

Intent Tab — Full Project Context for Agentic Development

Feed this entire document to your LLM agent (Cursor / Copilot / Claude Code) before starting any task.

It contains: project summary, current state, file-by-file breakdown, what's missing, and a prioritized task roadmap.

1. What Is This Project?

Intent Tab is a Chrome Extension (Manifest V3) that overrides the browser's new tab page.

Core philosophy: The browser destroys user intent through tab overload and distraction. Intent Tab acts as a lightweight behavioral layer — every new tab forces the user to declare what they're trying to do right now.

It is NOT:

- A task manager
- A note-taking app
- A focus blocker

It IS:

- A **cognitive continuity tool** — it preserves and displays the user's active goal throughout a browsing session, and gently nudges them when they drift.

2. Tech Stack

Layer	Technology
Frontend	Vanilla HTML, CSS, JavaScript (no framework)
Extension APIs	Chrome Extension Manifest V3
Storage	<code>chrome.storage.local</code>
Tab Monitoring	<code>chrome.tabs</code> API
Background	Service Worker (<code>background.js</code>)
Fonts	Custom fonts in <code>/fonts/</code> folder

No npm, no bundler, no React. Pure browser extension stack.

3. Current File Structure (What Actually Exists)

```
Intent-tab/
|
├─ manifest.json           ← Extension config (MV3)
├─ newtab.html             ← Main new tab override UI
├─ popup.html              ← Toolbar popup (extension icon click)
|
├─ css/
|   ├─ main.css            ← All UI styles
|   └─ animations.css      ← Ambient glow animations
|
├─ js/
|   ├─ newtab.js           ← Main controller for newtab page
|   ├─ storage.js          ← Read/write sessions to chrome.storage
|   ├─ tracker.js          ← Session timer logic
|   ├─ focus.js            ← Distraction detection (partial/stub)
|   ├─ utils.js            ← Shared helper functions
|   └─ background.js       ← SERVICE WORKER – referenced in manifest but
EMPTY/MISSING
|
├─ assets/
|   ├─ logo.png            ← MISSING (HTML references it but file doesn't
exist)
|   ├─ icon16.svg
|   ├─ icon48.svg
|   └─ icon128.svg
|
└─ fonts/                  ← Custom font files
```

4. File-by-File Breakdown

manifest.json

- Manifest Version: 3
- Overrides newtab with `newtab.html`
- Popup: `popup.html`
- Permissions: `storage`, `tabs`
- Background service worker: `js/background.js`
- Web accessible resources: `fonts/*`
- Icons: SVG format (16, 48, 128)

`newtab.html`

Main UI rendered on every new tab. Contains:

- Ambient glow background (3 colored glow pods: blue, indigo, cyan)
- Brand header with logo + "Intent Tab" text
- Hero greeting: "Hi User, what is your focus for this session?"
- Intent input form (`#intent-form`) with:
 - Text input for the intent (`#intent-input`)
 - Submit button (SVG arrow icon)
 - Secondary hidden input for category (`#category-input`), appears after intent is submitted)
- Google Search bar (native `<form>` posting to `google.com/search`)
- Session card (`#session-card`), hidden by default) with:
 - Category badge (`#session-category`)
 - Active status indicator
 - Intent title (`#session-intent`)
 - Session start time (`#session-start`)
 - Live timer (`#session-timer`)
 - End Session button (`#stop-button`)
- Scripts loaded: `storage.js` → `focus.js` → `utils.js` → `tracker.js` → `newtab.js`

`popup.html`

Rendered when user clicks extension icon in toolbar. Contains:

- Status block (`#popup-status`) — shows current intent if session is active
- "Open Intent Tab" button (`#open-new-tab`)
- "End Session" button (`#end-session`), hidden by default)
- Recent sessions history list (`#history-list`), hidden by default)
- Scripts loaded: `storage.js` → `focus.js` → `utils.js` → `popup.js`

`js/storage.js` (inferred from usage)

Handles all `chrome.storage.local` read/write operations. Expected functions:

- `saveSession(intentData)` — saves active session object
- `getSession(callback)` — retrieves current session
- `saveHistory(sessionData)` — appends completed session to history
- `getHistory(callback)` — retrieves past sessions array
- `clearSession()` — clears active session on end

`js/tracker.js` (inferred from usage)

Handles the live session timer.

Expected functions:

- `startTimer(startTime)` — begins counting from session start
- `stopTimer()` — clears interval
- Updates `#session-timer` element with HH:MM:SS format

`js/focus.js` (partially implemented)

Meant to detect distraction sites. Currently likely a stub.

Intended behavior:

- Maintains a list of distraction domains (YouTube, Instagram, Reddit, Twitter, etc.)
- Checks tab URL against this list
- Triggers a non-blocking nudge notification

`js/newtab.js` (inferred from HTML)

Main controller for the newtab page.

Expected behavior:

- Listens for `#intent-form` submit
- Saves intent + category via `storage.js`
- Shows `#session-card` and hides intent form
- Starts `tracker.js` timer
- On `#stop-button` click: ends session, saves to history, resets UI
- On page load: checks if active session exists and restores UI state

`js/background.js` — CRITICAL MISSING FILE

Service worker. Currently empty or missing. This is the **most important missing piece** for tab monitoring and distraction detection. Must implement:

- `chrome.tabs.onUpdated` listener
- URL checking against distraction list
- Sending message to content/newtab page when distraction detected

5. Known Bugs / Issues to Fix First

#	Issue	File	Priority
1	<code>background.js</code> is empty — extension will throw service	<code>js/background.js</code>	●

#	Issue	File	Priority
	worker error on load		Critical
2	<code>assets/logo.png</code> is missing — broken image in header	<code>newtab.html</code> + <code>assets/</code>	Critical
3	<code>focus.js</code> distraction detection is not wired to <code>background.js</code>	<code>js/focus.js</code>	High
4	<code>popup.js</code> likely missing (popup.html references it)	<code>js/popup.js</code>	Critical
5	Greeting hardcoded as "Hi User" — should use stored name or be generic	<code>newtab.html</code>	Low

6. Feature Roadmap (Prioritized)

Phase 1 — Fix & Complete MVP (Do This First)

Task 1.1 — Create `js/background.js` (Service Worker)

Goal: Monitor tab URLs and detect distraction sites.

Implementation:

- Listen to `chrome.tabs.onUpdated` event

- Maintain `DISTRACTION_DOMAINS` array: `['youtube.com', 'instagram.com', 'reddit.com', 'twitter.com', 'x.com', 'facebook.com', 'tiktok.com']`

- On URL match: check if active session exists via `chrome.storage.local`

- If session active + distraction detected: send `chrome.runtime.sendMessage({ type: 'DISTRACTION_DETECTED', domain: matchedDomain })`

- `newtab.js` and `popup.js` should listen for this message and show a toast

File: `js/background.js`

Task 1.2 — Create `js/popup.js`

Goal: Power the popup UI.

Implementation:

- On load: read active session from storage.js
- If session active: show intent text in #popup-status, show #end-session button, show #history-summary
- #open-new-tab click: chrome.tabs.create({ url: 'chrome://newtab' })
- #end-session click: call storage.clearSession(), update UI
- Populate #history-list with last 5 sessions from getHistory()

File: js/popup.js

Task 1.3 — Wire Distraction Toast in `newtab.js`

Goal: Show non-blocking nudge when background.js sends DISTRACTION_DETECTED message.

Implementation:

- Add chrome.runtime.onMessage listener in newtab.js
- On DISTRACTION_DETECTED: create a toast div with message like:
"You opened [domain] while working on: [current intent]"
- Toast auto-dismisses after 5 seconds
- No blocking, no redirects — just awareness

File: js/newtab.js (add to existing)

Task 1.4 — Fix missing `assets/logo.png`

Goal: Either create a simple SVG logo or change the img tag to use icon128.svg

Option A (recommended): Change newtab.html line:

```
  
to:  

```

Option B: Create a simple text-based SVG logo and save as assets/logo.svg, update reference.

File: newtab.html

Phase 2 — Core Feature Upgrades

Task 2.1 — Analytics Dashboard Page

Goal: Show daily/weekly breakdown of session data.

New files needed:

- analytics.html
- js/analytics.js
- css/analytics.css (or extend main.css)

Data to show:

- Today's sessions: intent name + duration
- Total focus time today
- Total distraction count today
- Last 7 days bar chart (use Chart.js CDN – no install needed)

Storage schema needed in storage.js:

```
{
  sessions: [
    {
      intent: "Study DBMS Unit 3",
      category: "Study",
      startTime: "2026-06-09T10:00:00",
      endTime: "2026-06-09T10:42:00",
      duration: 2520, // seconds
      distractions: 2
    }
  ]
}
```

Add "View Analytics" link in popup.html that opens analytics.html.

Add to manifest.json web_accessible_resources.

Task 2.2 — User-Configurable Distraction List

Goal: Let users add/remove their own distraction domains.

New files needed:

- settings.html
- js/settings.js

Storage key: 'distractionDomains' → string array

UI: Simple list with add input + remove buttons

Default list preloaded on first install.

Add "Settings" link in popup.html.

background.js should read from storage instead of hardcoded array.

Task 2.3 — Tab Grouping via Chrome Tab Groups API

Goal: Auto-group tabs opened after setting intent.

Implementation in background.js:

- On new tab created after session start: add to a tab group named after the intent
- Use `chrome.tabGroups.update()` to color-code by category:
 - Study → blue
 - Dev → green
 - Market → orange
 - Default → grey

New permission needed in manifest.json: "tabGroups"

Note: Chrome 89+ only. Add graceful fallback check.

Phase 3 — AI Enhancement Layer

Task 3.1 — AI Intent Suggestions

Goal: Suggest completions as user types in intent input.

Approach: Call a free/cheap LLM API (Gemini Flash or Claude Haiku) on keyup (debounced 400ms).

Prompt template:

```
"The user is setting a focus intent for their browser session.  
They typed: '[partial input]'  
Past intents: [last 5 intents from history]  
Suggest 3 short, specific completions. Return as JSON array."
```

Show suggestions as a small dropdown below the input.
Click to autofill.

API key storage: `chrome.storage.local` (user enters it in `settings.html`)

Task 3.2 — Smart Distraction Nudge Messages

Goal: Replace hardcoded toast messages with AI-generated ones.

When distraction detected, call LLM with:
"User intended to: [intent]. They just opened [domain].
Write a single short, non-judgmental, friendly reminder (max 12 words)."

Cache response per (intent + domain) pair to avoid repeated API calls.

7. Data Schema Reference

Active Session Object (in `chrome.storage.local` key: `activeSession`)

```
json
{
  "intent": "Study DBMS Unit 3",
  "category": "Study",
  "startTime": "2026-06-09T10:00:00.000Z",
  "tabId": 42
}
```

Session History Array (in `chrome.storage.local` key: `sessionHistory`)

```
json
[
  {
    "intent": "Study DBMS Unit 3",
    "category": "Study",
    "startTime": "2026-06-09T10:00:00.000Z",
    "endTime": "2026-06-09T10:42:00.000Z",
    "duration": 2520,
    "distractions": 2
  }
]
```

8. Manifest Permissions Needed Per Phase

Phase	Permission to Add
Phase 1 (current)	<code>storage</code> , <code>tabs</code> — already there

Phase	Permission to Add
Phase 2 Tab Groups	Add <code>"tabGroups"</code>
Phase 2 Notifications	Add <code>"notifications"</code> (optional, for OS-level nudge)
Phase 3 AI	No new permissions needed (fetch to external API is allowed)

9. Message Passing Architecture (background.js ↔ newtab.js)

```
background.js (Service Worker)
└─ chrome.runtime.sendMessage({ type: 'DISTRACTION_DETECTED', domain:
'youtube.com' })
    ↓
newtab.js / popup.js
└─ chrome.runtime.onMessage.addListener((msg) => {
    if (msg.type === 'DISTRACTION_DETECTED') showToast(msg.domain)
  })
```

10. Agent Instructions — How to Work on This

When implementing any task:

1. **Read the file first** before editing — understand existing structure.
2. **Do not add any npm dependencies** — vanilla JS only, CDN links allowed in HTML.
3. **Keep chrome.storage calls async** — always use callbacks or async/await with `chrome.storage.local.get/set`.
4. **Service worker limitations** — `background.js` cannot access DOM. Use message passing only.
5. **Test by loading unpacked** — `chrome://extensions` → Developer Mode → Load Unpacked → select repo root.
6. **Never use `localStorage`** — always use `chrome.storage.local` for data that must persist across tabs.
7. **Background.js is a service worker** — it can wake up and sleep. Do not rely on in-memory variables for persistent state. Always read from `chrome.storage.local`.

11. Start Order (Recommended for First Session)

Step 1: Create `js/background.js` with tab URL monitoring

Step 2: Create js/popup.js to power the popup
Step 3: Add distraction toast to js/newtab.js
Step 4: Fix assets/logo.png reference in newtab.html
Step 5: Test full flow end-to-end by loading as unpacked extension
Step 6: Then move to Phase 2 features

Document generated for agentic development use. Repo: <https://github.com/always-yash/Intent-tab>